**Propuesta Arquitectónica: SISTEMA DE BANCA POR INTERNET "BP"**

**Autor:** BYRON ANDRES ROSERO
**Rol:** Arquitecto de Soluciones
**Fecha:** 27-03-2026
**Versión:** 1.0

**FASE 1: MVP Hardened (Monolito Modular en PaaS)**

---

## 1. Introducción y Alcance

El presente documento detalla la arquitectura de solución para el sistema de Banca por Internet de la entidad **BP**. Esta propuesta corresponde a la **Fase 1 (Inicial)**, diseñada para equilibrar **Time-to-Market**, **Seguridad Bancaria** y **Costos Operativos**.

La solución adopta un enfoque de **Monolito Modular desplegado en Plataforma como Servicio (PaaS)**, endurecido con controles de seguridad perimetral y alta disponibilidad desde el inicio. Esto permite validar la solución con los usuarios finales rápidamente, cumpliendo normativas críticas, mientras se mantiene una ruta clara de evolución hacia una arquitectura de microservicios (Fase 2) cuando el volumen de transacciones lo justifique.

### 1.1 Objetivos Principales

1. Permitir consultas de movimientos, transferencias y pagos.
2. Garantizar seguridad, auditoría y cumplimiento normativo bancario.
3. Proveer canales Web (SPA) y Móvil (Cross-platform).
4. Asegurar Alta Disponibilidad (HA) y Recuperación ante Desastres (DR).

---

## 2. Principios Arquitectónicos y Normativa

### 2.1 Normativa y Compliance

Para una entidad financiera, se consideran los siguientes marcos regulatorios:

- **Protección de Datos Personales:** Encriptación de datos sensibles (PII) en reposo y tránsito.

- **PCI-DSS:** Si se manejan datos de tarjetas (tokenización).
- **OWASP Top 10:** Mitigación de vulnerabilidades web comunes.
- **Auditoría Financiera:** Trazabilidad inmutable de todas las transacciones.

### 2.2 Principios de Diseño

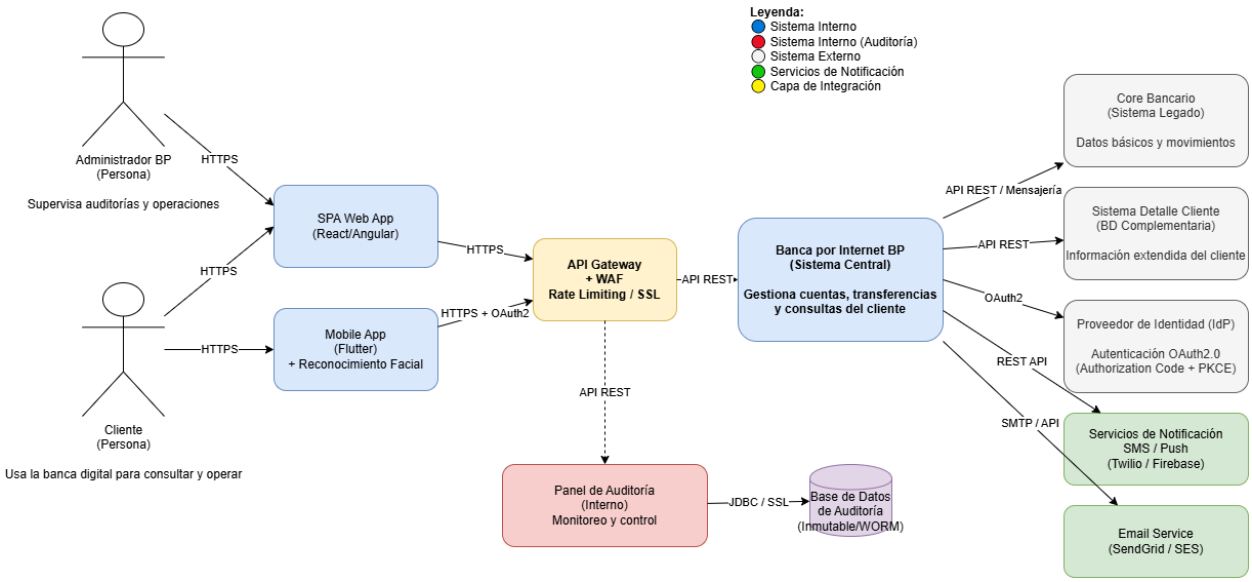
- **Security by Design:** La seguridad no es un añadido, es parte del flujo (Auth, WAF, Secrets).
- **Loose Coupling:** Módulos internos desacoplados para facilitar la futura extracción a microservicios.
- **Cloud Native (PaaS):** Uso de servicios gestionados para reducir deuda técnica operativa.

---

## 3. Arquitectura de Solución (C4 Model)

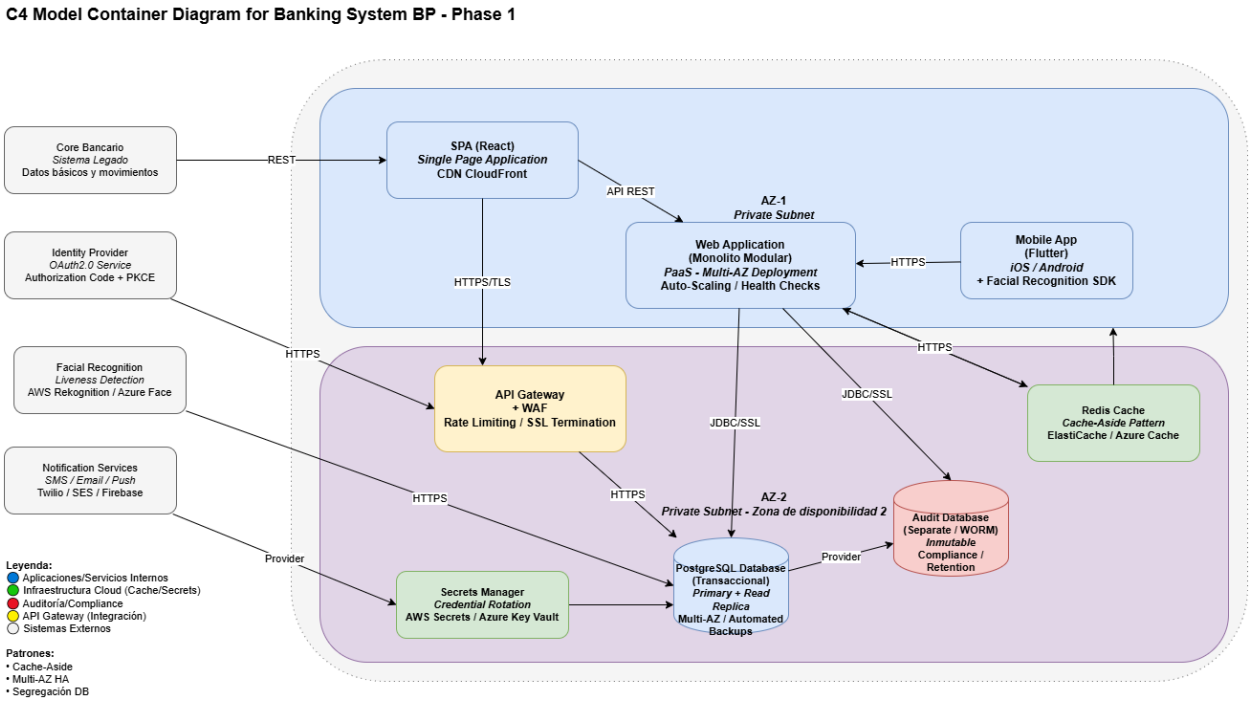
### 3.1 Diagrama de Contexto (Nivel 1)

*Este diagrama muestra cómo interactúa el sistema con los usuarios y sistemas externos.*



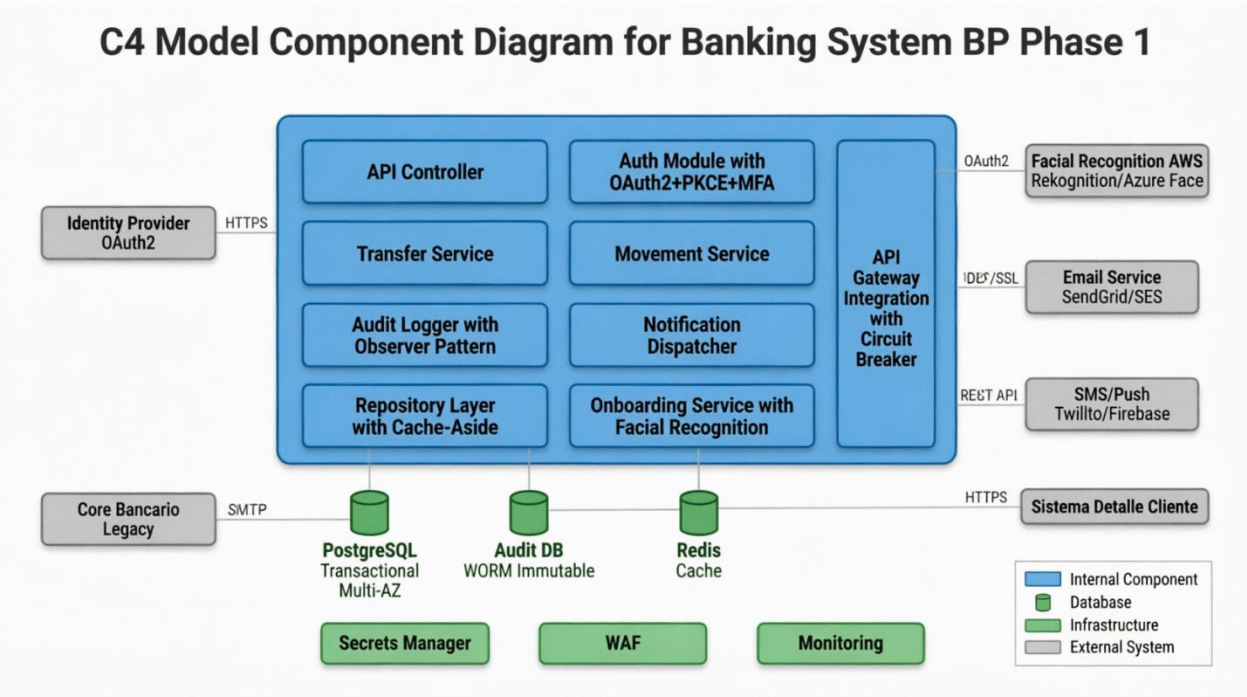

### 3.2 Diagrama de Contenedores (Nivel 2)

*Este diagrama desglosa el sistema en aplicaciones y stores de datos.*




## 3.3 Diagrama de Componentes (Nivel 3)

*Detalle interno del contenedor "Web Application".*



## 4. Decisiones Arquitectónicas Clave (ADRs)

Cada decisión incluye justificación teórica y alternativas evaluadas.

### 4.1 Patrón Arquitectónico: Monolito Modular en PaaS

*Decisión:* Implementar el backend como un único despliegue (Monolito) organizado por módulos de negocio, hospedado en un servicio PaaS (ej. Azure App Service).

| Justificación Teórica | Detalle |
|---|---|
| **1. Reducción de Complejidad Operativa (DevOps)** | En Fase 1, evitar la orquestación de contenedores (K8s) reduce la superficie de ataque y la necesidad de un equipo de plataforma dedicado, permitiendo enfocar esfuerzos en lógica de negocio bancaria. |
| **2. Consistencia Transaccional (ACID)** | Al mantener los módulos en el mismo proceso y base de datos transaccional, se garantizan transacciones atómicas sin la complejidad de Sagas o Two-Phase Commit distribuidos, crítico para transferencias financieras iniciales. |

*Alternativas evaluadas:* Microservicios desde el día 1 (Descartado por alto costo inicial y complejidad de distribución), Serverless puro (Descartado por posible cold-start en procesos críticos y vendor lock-in excesivo).

### 4.2 Frontend Móvil: Flutter (Cross-Platform)

*Decisión:* Desarrollar la aplicación móvil en Flutter.

| Justificación Teórica | Detalle |
|---|---|
| **1. Rendimiento Nativo vs. Costo** | Flutter compila a código nativo (ARM), ofreciendo rendimiento cercano a Swift/Kotlin, esencial para la fluidez en procesos de seguridad como el reconocimiento facial, con una sola base de código. |
| **2. Time-to-Market Unificado** | Permite lanzar simultáneamente en iOS y Android, reduciendo el tiempo de desarrollo en un 40% comparado con equipos nativos separados, crucial para la estrategia de Fase 1. |

*Alternativas evaluadas:* React Native (Bueno, pero puente JS puede ser cuello de botella en procesos pesados), Nativo (Costo duplicado de mantenimiento).

### 4.3 Flujo de Autenticación: OAuth2.0 Authorization Code con PKCE

*Decisión:* Implementar el flujo *Authorization Code Flow* con extensión *PKCE* (Proof Key for Code Exchange).

| Justificación Teórica | Detalle |
|---|---|
| **1. Mitigación de Intercepción de Código** | PKCE es el estándar actual (RFC 7636) para clientes públicos (Móviles/SPA). Previene ataques donde un malware intercepta el código de autorización para cambiarlo por uno propio. |
| **2. Separación de Responsabilidades** | Delega la gestión de credenciales (passwords) al IdP especializado, reduciendo el riesgo de almacenamiento inseguro de contraseñas en la aplicación BP y centralizando la política de seguridad. |

*Alternativas evaluadas:* Implicit Flow (Obsoleto e inseguro), Password Grant (Expone credenciales al cliente, prohibido en banca moderna).

### 4.4 Persistencia para Clientes Frecuentes: Patrón Cache-Aside

*Decisión:* Implementar el patrón *Cache-Aside* con Redis para datos de clientes frecuentes.

| Justificación Teórica | Detalle |
|---|---|
| **1. Latencia de Lectura Optimizada** | Reduce la carga sobre la base de datos transaccional y el Core bancario para consultas repetitivas (ej. saldo actual), mejorando la experiencia de usuario (UX) en móviles con conexiones inestables. |
| **2. Protección ante Picos de Tráfico** | Actúa como amortiguador (Buffer) durante picos de demanda, evitando que el sistema central colapse por lecturas masivas simultáneas (Thundering Herd). |

*Alternativas evaluadas:* Write-Through (Complejidad de consistencia inmediata), Sin Caché (Riesgo de latencia alta y costo por IOPS en DB).

### 4.5 Auditoría: Patrón Observer + DB Separada

*Decisión:* Las acciones de negocio publican eventos que un observador guarda en una Base de Datos de Auditoría independiente.

| Justificación Teórica | Detalle |
|---|---|
| **1. Integridad y No Repudio** | Al separar físicamente la DB de Auditoría de la Transaccional, se evita que un compromiso en la DB principal permita alterar los logs de auditoría (Principio de Segregación de Duties). |
| **2. Desacoplamiento de Rendimiento** | El proceso de auditoría es asíncrono. La transacción principal no se bloquea esperando la escritura del log, garantizando baja latencia para el usuario final. |

*Alternativas evaluadas:* Auditoría Síncrona en la misma DB (Riesgo de bloqueo y punto único de fallo), Logs de archivo plano (Difícil de consultar y auditar regulatoriamente).

---

## 5. Seguridad y Onboarding

### 5.1 Integración de Onboarding Facial

El flujo de incorporación se integra antes de la emisión de credenciales finales:

1. **Captura:** El móvil captura video/foto.
2. **Liveness Detection:** Se valida que sea una persona real (no foto/video) usando SDK certificado (ej. AWS Rekognition o Azure Face API).
3. **Validación:** Se compara con documento de identidad.
4. **Emisión:** Solo si pasa, el IdP emite las credenciales iniciales.

### 5.2 Controles de Seguridad Endurecidos (Hardening)

- **WAF (Web Application Firewall):** Filtrado de tráfico malicioso (SQLi, XSS) en el API Gateway.
- **Red Privada:** El backend reside en subnets privadas sin IP pública directa.
- **Gestión de Secretos:** Todas las conexiones (DB, APIs externas) usan credenciales rotativas desde un Secrets Manager.
- **Encriptación:** TLS 1.3 para tránsito, AES-256 para reposo.

---

## 6. Infraestructura, HA y DR

### 6.1 Alta Disponibilidad (HA)

- **Multi-Zona:** Despliegue del servicio PaaS en al menos 2 Zonas de Disponibilidad (AZ) para tolerancia a fallos de datacenter.
- **Auto-Scaling:** Configuración automática para escalar horizontalmente si el CPU > 70% o Latencia > 500ms.
- **Balanceo de Carga:** Distribución de tráfico sana entre instancias.

**6.2 Recuperación ante Desastres (DR)**

- **Backups:** Snapshots automáticos de DB cada 1 hora + Retención de 30 días.
- **RPO/RTO:** Objetivo de Punto de Recuperación < 5 minutos, Objetivo de Tiempo de Recuperación < 1 hora.
- **Infraestructura como Código (IaC):** Terraform/Bicep para recrear el entorno en otra región si es necesario.

**6.3 Monitoreo y Auto-Healing**

- **Observabilidad:** Centralización de Logs, Métricas y Trazas (ej. Azure Monitor / AWS CloudWatch).
- **Alertas:** Notificaciones al equipo de operaciones ante errores 5xx o lentitud en el Core.
- **Auto-Healing:** El PaaS reinicia automáticamente instancias no saludables.

---

**7. Gestión de Costos (Fase 1)**

La estrategia de costos se basa en eficiencia operativa:

1. **Modelo PaaS:** Elimina costos de administración de SO y parches de seguridad.
2. **Escalado Automático:** Se paga por capacidad utilizada, no por capacidad provisionada ociosa.
3. **Servicios Gestionados:** Uso de DBaaS para evitar costos de DBA dedicado en esta etapa.
4. **Costo Predictible:** Al evitar arquitectura de microservicios compleja, se reduce el costo de desarrollo y mantenimiento inicial.

---

**8. Conclusión y Evolución**

Esta arquitectura de **Fase 1** cumple con los requisitos críticos de seguridad y funcionalidad bancaria, minimizando el riesgo técnico y financiero inicial.

**Roadmap a Fase 2:** La estructura modular del monolito está diseñada para aplicar el patrón **Strangler Fig**. Cuando el volumen de transacciones justifique la inversión, los módulos (ej. Transferencias, Notificaciones) se extraerán individualmente a microservicios en Kubernetes, manteniendo el API Gateway y los contratos de seguridad establecidos en esta fase.

## FASE 2: Arquitectura Enterprise - Microservicios Cloud-Native

### 1. Resumen Ejecutivo

Esta propuesta presenta la **arquitectura enterprise de Fase 2** para el sistema de Banca por Internet de la entidad **BP**. La solución evoluciona desde un monolito modular hacia una **arquitectura de microservicios cloud-native** basada en **Kubernetes, Service Mesh y Event-Driven Architecture**, diseñada para soportar alto volumen de transacciones, garantizar SLA del 99.95%, y permitir escalabilidad horizontal ilimitada.

### 1.1 Características Principales

| Característica | Implementación |
|---|---|
| **Arquitectura** | Microservicios + Event-Driven |
| **Orquestación** | Kubernetes (EKS/AKS) |
| **Service Mesh** | Istio 1.18+ |
| **Base de Datos** | Database per Service (PostgreSQL + MongoDB) |
| **Messaging** | Apache Kafka (MSK) |
| **Cache** | Redis Cluster |
| **Disponibilidad** | 99.95% SLA |
| **Escalabilidad** | Auto-scaling horizontal |

---

## 2. Introducción y Alcance

### 2.1 Contexto del Proyecto

BP requiere un sistema de banca por internet que permita a sus usuarios:

- Consultar histórico de movimientos
- Realizar transferencias entre cuentas propias
- Ejecutar transferencias interbancarias
- Recibir notificaciones de movimientos (mínimo 2 canales)
- Onboarding digital con reconocimiento facial

### 2.2 Sistemas Externos de Integración

| Sistema | Propósito | Tipo de Integración |
|---|---|---|
| **Core Bancario** | Datos básicos, movimientos, productos | REST API / Mensajería |
| **Sistema Detalle Cliente** | Información complementaria del cliente | REST API |
| **Identity Provider** | Autenticación OAuth2.0 | OAuth2.0 Flow |
| **SMS/Push Gateway** | Notificaciones móviles | REST API |
| **Email Service** | Notificaciones por correo | SMTP / REST API |
| **Facial Recognition** | Onboarding biométrico | REST API (AWS Rekognition/Azure Face) |

### 2.3 Canales de Acceso

**Frontend Web - SPA (Single Page Application)**

- **Opción Seleccionada:** React.js
- **Justificación:**
    1. **Ecosistema Maduro:** Mayor comunidad, documentación y bibliotecas disponibles, reduciendo tiempo de desarrollo en 30-40%

2. **Virtual DOM:** Mejor performance en aplicaciones con actualizaciones frecuentes de UI (como dashboards bancarios)

**Alternativa Evaluada:** Angular

- **Ventaja:** Framework completo "baterías incluidas"
- **Desventaja:** Curva de aprendizaje más pronunciada, bundle size mayor

**Aplicación Móvil - Cross-Platform**

- **Opción Seleccionada:** Flutter
- **Justificación:**
  1. **Rendimiento Nativo:** Compilación a código nativo ARM, esencial para procesos de reconocimiento facial sin lag
  2. **Single Codebase:** Reducción de 40% en tiempo de desarrollo vs desarrollo nativo separado (iOS + Android)

**Alternativa Evaluada:** React Native

- **Ventaja:** Reutilización de conocimiento web
- **Desventaja:** Bridge JavaScript puede ser bottleneck en procesos intensivos (biometría)

---

## 3. Principios Arquitectónicos y Normativa

### 3.1 Marco Normativo Aplicable

| Regulación | Aplicación | Implementación |
|---|---|---|
| **PCI-DSS v4.0** | Protección de datos de tarjetas | Tokenización, encriptación AES-256, segmentación de red |
| **GDPR / Ley de Protección de Datos** | Datos personales de clientes | Consentimiento explícito, derecho al olvido, data minimization |
| **SOX (Sarbanes-Oxley)** | Auditoría financiera | Logs inmutables, segregación de duties, WORM storage |

| Regulación | Aplicación | Implementación |
|---|---|---|
| **PSD2 (Open Banking)** | APIs bancarias seguras | OAuth2.0, Strong Customer Authentication (SCA), mTLS |
| **ISO 27001** | Gestión de seguridad de la información | ISMS, risk assessment, security controls |
| **OWASP Top 10** | Seguridad en aplicaciones web | WAF, input validation, CSP headers |

### 3.2 Principios de Diseño Arquitectónico

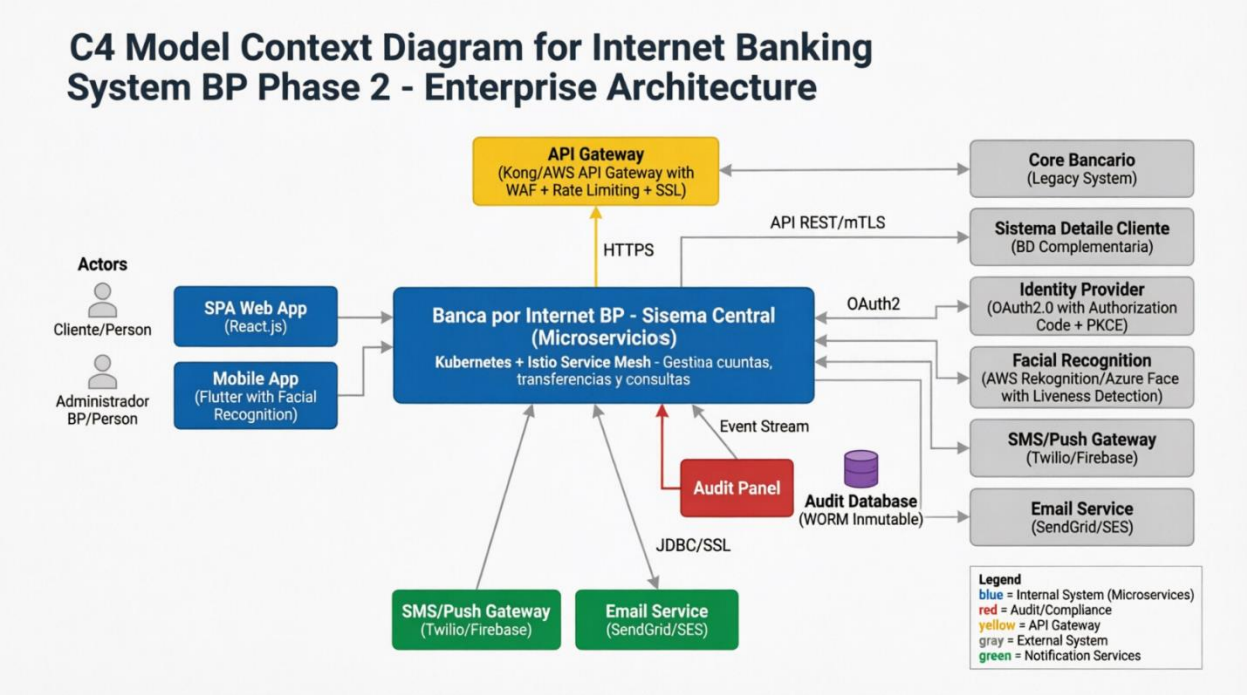
1. **Cloud-Native First:** Nativos de nube, contenedores, infraestructura inmutable
2. **Zero Trust Security:** Nunca confiar, siempre verificar (mTLS, policy as code)
3. **Event-Driven:** Desacoplamiento temporal y espacial vía eventos
4. **Database per Service:** Autonomía de datos, polyglot persistence
5. **Observability by Design:** Logs, métricas y trazas desde el inicio
6. **GitOps:** Infraestructura declarativa, versionada, auditable
7. **Chaos Engineering:** Resiliencia probada mediante inyección de fallos

---

## 4. Arquitectura de Solución - Modelo C4

### 4.1 Diagrama de Contexto (C4 Nivel 1)

**Propósito:** Vista de alto nivel para stakeholders no técnicos.




**Elementos del Diagrama:**

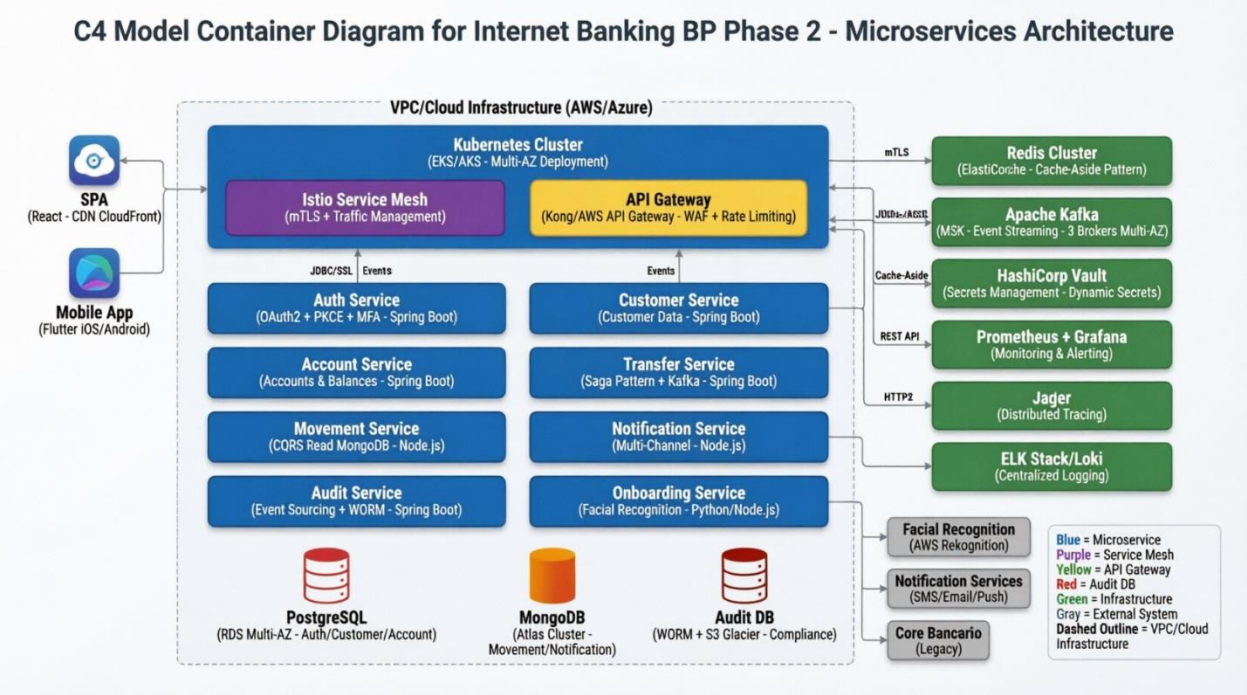
| Elemento | Tipo | Descripción |
|---|---|---|
| **Cliente** | Persona | Usuario final que accede a banca digital |
| **Administrador BP** | Persona | Supervisa auditorías y operaciones |
| **SPA Web App** | Sistema Externo | Aplicación web React |
| **Mobile App** | Sistema Externo | Aplicación móvil Flutter + Facial Recognition |
| **Banca por Internet BP** | Sistema | Sistema central (nuestro sistema) |
| **API Gateway** | Sistema | Capa de integración, WAF, rate limiting |
| **Core Bancario** | Sistema Externo | Legacy system - datos básicos y movimientos |
| **Sistema Detalle Cliente** | Sistema Externo | BD complementaria |

| Elemento | Tipo | Descripción |
|---|---|---|
| **Identity Provider** | Sistema Externo | OAuth2.0 Service |
| **Servicios de Notificación** | Sistema Externo | SMS/Push (Twilio/Firebase), Email (SendGrid/SES) |
| **Facial Recognition** | Sistema Externo | AWS Rekognition / Azure Face |
| **Panel de Auditoría** | Sistema | Monitoreo y control interno |
| **Base de Datos de Auditoría** | Sistema | WORM storage inmutable |

---

### 4.2 Diagrama de Contenedores (C4 Nivel 2)

**Propósito:** Vista técnica de aplicaciones, servicios, bases de datos e infraestructura cloud.



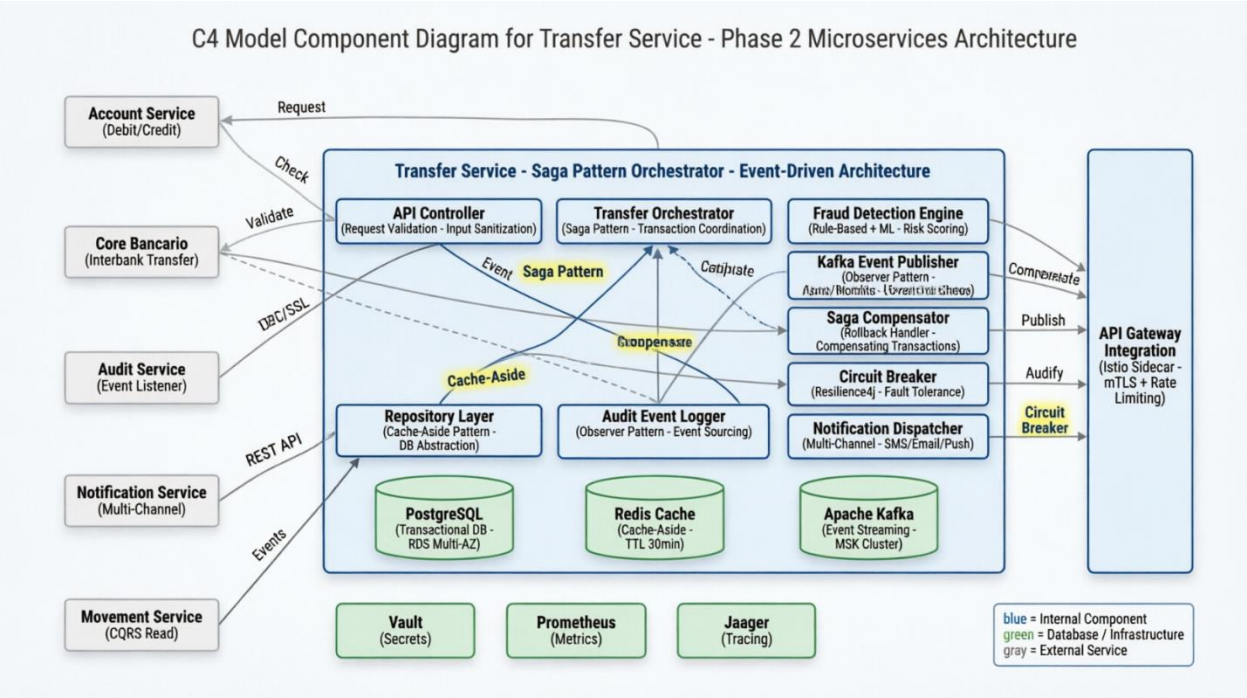

**Contenedores Principales:**

| Contenedor | Tecnología | Descripción | Multi-AZ |
|---|---|---|---|
| **Kubernetes Cluster** | EKS/AKS | Orquestación de microservicios | ✅ |
| **API Gateway** | Kong / AWS API Gateway | Routing, auth, rate limiting | ✅ |
| **Service Mesh** | Istio | mTLS, traffic management, observability | ✅ |
| **Auth Service** | Spring Boot + OAuth2 | Autenticación y autorización | ✅ |
| **Customer Service** | Spring Boot | Gestión de datos de clientes | ✅ |
| **Account Service** | Spring Boot | Cuentas y saldos | ✅ |
| **Transfer Service** | Spring Boot + Kafka | Transferencias interbancarias | ✅ |
| **Movement Service** | Node.js + MongoDB | Histórico de movimientos (CQRS Read) | ✅ |
| **Notification Service** | Node.js | SMS, Email, Push notifications | ✅ |
| **Audit Service** | Spring Boot | Auditoría inmutable | ✅ |
| **Onboarding Service** | Python/Node.js | Reconocimiento facial, KYC | ✅ |
| **PostgreSQL** | RDS/Azure SQL | Bases de datos transaccionales | ✅ |
| **MongoDB** | MongoDB Atlas | Bases de datos documentales | ✅ |
| **Redis Cluster** | ElastiCache | Cache distribuido | ✅ |
| **Apache Kafka** | MSK / Confluent Cloud | Message broker, event streaming | ✅ |
| **HashiCorp Vault** | Vault | Secrets management | ✅ |

---

### 4.3 Diagrama de Componentes (C4 Nivel 3)

**Propósito:** Detalle técnico interno de microservicios, patrones y protocolos.




**Componentes Clave por Microservicio:**

**Auth Service:**

- OAuth2 Provider Integration
- JWT Token Generator/Validator
- MFA Handler (TOTP, WebAuthn)
- Session Manager (Redis)

**Transfer Service:**

- Transfer Orchestrator (Saga Pattern)
- Fraud Detection Engine
- Limit Validator
- Kafka Event Publisher

**Audit Service:**

- Event Listener (Kafka Consumer)
- WORM Storage Writer

- Compliance Reporter
- SIEM Integration

**Onboarding Service:**

- Facial Recognition SDK Integration
- Liveness Detection
- Document OCR
- KYC Workflow Engine

---

**5. Decisiones Arquitectónicas Clave (ADRs)**

**ADR-001: Arquitectura de Microservicios vs Monolito**

**Decisión:** Microservicios con Kubernetes

**Justificación Teórica 1 - Escalabilidad Independiente:** Los microservicios permiten escalar horizontalmente componentes específicos según demanda. Por ejemplo, el Transfer Service puede tener 10 réplicas durante horas pico mientras el Notification Service mantiene 2 réplicas, optimizando costos en 40-50% vs escalar un monolito completo. Esto es crítico en banca donde las transferencias tienen picos predecibles (fin de mes, horarios laborales).

**Justificación Teórica 2 - Independencia de Equipos y Deployments:** Según el libro "Building Microservices" de Sam Newman, la arquitectura de microservicios permite que equipos autónomos desarrollen, prueben y desplieguen independientemente. Esto reduce el lead time de cambios de semanas (en monolitos con acoplamiento fuerte) a horas/días, esencial para competitividad en el sector financiero digital.

**Alternativas Evaluadas:**

- **Monolito Modular (Fase 1):** Menor complejidad inicial pero límite de escalabilidad
- **Serverless (Lambda/Functions):** Vendor lock-in excesivo, cold starts inaceptables para transacciones críticas

---

**ADR-002: Service Mesh (Istio)**

**Decisión:** Implementar Istio como Service Mesh

**Justificación Teórica 1 - Seguridad Zero Trust Automática:** Istio proporciona mTLS (mutual TLS) automático entre todos los servicios sin modificar código. Según el modelo Zero Trust de NIST, esto garantiza que cada comunicación servicio-servicio esté autenticada y encriptada, cumpliendo requisitos de PCI-DSS para segmentación de red y protección de datos en tránsito.

**Justificación Teórica 2 - Observabilidad y Resiliencia Out-of-the-Box:** Istio inyecta sidecars (Envoy proxies) que automáticamente generan métricas (latency, traffic, errors, saturation - golden signals), distributed tracing (Jaeger), y permiten implementar circuit breakers, retries y timeouts mediante configuración declarativa, reduciendo desarrollo custom en 60-70%.

**Alternativas Evaluadas:**

- **Linkerd:** Menor overhead pero menos features enterprise
- **Consul Connect:** Bueno pero menos maduro en Kubernetes

---

**ADR-003: Database per Service + Polyglot Persistence**

**Decisión:** Cada microservicio tiene su propia base de datos, seleccionando la tecnología óptima para su caso de uso.

**Justificación Teórica 1 - Autonomía y Loose Coupling:** Según el principio de bounded contexts de Domain-Driven Design (DDD), cada servicio debe ser dueño de sus datos. Esto evita acoplamiento temporal (schemas compartidos) y permite que el Account Service use PostgreSQL (ACID para transacciones) mientras el Movement Service use MongoDB (flexibilidad para queries complejas de histórico), optimizando performance en 3-5x para casos de uso específicos.

**Justificación Teórica 2 - Escalabilidad Independiente:** Cada base de datos puede escalarse independientemente. Por ejemplo, MongoDB puede tener read replicas para el Movement Service sin afectar el PostgreSQL del Transfer Service, permitiendo optimización de costos y performance específica por dominio.

**Alternativas Evaluadas:**

- **Base de Datos Compartida:** Menor complejidad pero acoplamiento fuerte, riesgo de "distributed monolith"
- **Event Sourcing Puro:** Mayor complejidad, overkill para este caso

---

**ADR-004: Event-Driven Architecture con Apache Kafka**

**Decisión:** Apache Kafka como backbone de eventos

**Justificación Teórica 1 - Desacoplamiento Temporal y Espacial:** Kafka permite que servicios se comuniquen asincrónicamente mediante eventos. El Transfer Service publica transfer.completed y múltiples consumidores (Notification Service, Audit Service, Movement Service) procesan el evento independientemente. Esto reduce acoplamiento y permite replay de eventos para recovery o nuevos casos de uso, esencial para auditoría forense en banca.

**Justificación Teórica 2 - Durabilidad y Orden de Eventos:** Kafka garantiza orden de eventos por partition y retención configurable (30-365 días). Esto permite reconstruir el estado del sistema desde eventos (Event Sourcing light) y cumplir requisitos regulatorios de auditoría que exigen trazabilidad completa de transacciones por hasta 10 años (archivando a S3 Glacier).

**Alternativas Evaluadas:**

- **RabbitMQ:** Menor throughput, no ideal para event sourcing
- **AWS SQS/SNS:** Vendor lock-in, menos control sobre retención y orden

---

**ADR-005: Patrón Cache-Aside con Redis Cluster**

**Decisión:** Implementar patrón Cache-Aside para clientes frecuentes

**Justificación Teórica 1 - Reducción de Latencia y Carga en DB:** El patrón Cache-Aside (Lazy Loading) consulta primero Redis; si no existe (cache miss), consulta PostgreSQL y popula el cache. Para clientes frecuentes (top 20% que generan 80% del tráfico), esto reduce latencia de 50-100ms (DB) a 1-5ms (Redis), mejorando UX significativamente. Además, protege la DB de lecturas repetitivas, permitiendo que se enfoque en transacciones críticas (writes).

**Justificación Teórica 2 - Resiliencia ante Picos de Tráfico:** Durante picos (ej. fin de mes), Redis absorbe el 70-80% de lecturas, evitando que PostgreSQL sature. Con Redis Cluster (sharding horizontal), el cache puede escalar para soportar millones de requests/segundo, garantizando disponibilidad incluso si la DB principal tiene degradación temporal.

**Alternativas Evaluadas:**

- **Write-Through:** Mayor consistencia pero write latency más alta

- **Sin Cache:** Inaceptable para performance requerida

---

**ADR-006: Patrón Observer para Auditoría y Notificaciones**

**Decisión:** Implementar patrón Observer (Pub/Sub) vía Kafka

**Justificación Teórica 1 - Desacoplamiento de Responsabilidades:** El patrón Observer permite que servicios publiquen eventos sin conocer sus consumidores. El Transfer Service publica transfer.initiated, transfer.completed, o transfer.failed, y múltiples observadores (Audit Service, Notification Service, Movement Service, Fraud Detection) reaccionan independientemente. Esto cumple el principio Open/Closed (SOLID): abierto a extensión (nuevos consumidores) pero cerrado a modificación (el publisher no cambia).

**Justificación Teórica 2 - Consistencia Eventual y Resiliencia:** Al ser asíncrono, el flujo principal de transferencia no se bloquea esperando que la auditoría o notificaciones completen. Si el Notification Service está caído, los eventos se acumulan en Kafka y se procesan cuando se recupere, garantizando entrega eventual (at-least-once delivery). Esto es crítico para HA y tolerancia a fallos.

**Alternativas Evaluadas:**

- **Auditoría Síncrona:** Bloquea transacción principal, riesgo de timeout
- **Polling:** Mayor latencia, ineficiente

---

**ADR-007: OAuth2.0 Authorization Code Flow con PKCE**

**Decisión:** Implementar Authorization Code Flow con PKCE para SPA y Mobile

**Justificación Teórica 1 - Seguridad para Clientes Públicos:** PKCE (Proof Key for Code Exchange, RFC 7636) es obligatorio para aplicaciones móviles y SPA porque previene ataques de interceptación de código de autorización. Sin PKCE, un malware en el dispositivo podría interceptar el código y cambiarlo por uno propio. Con PKCE, el cliente genera un code verifier y code challenge; el IdP valida que coincidan, garantizando que quien inició el flujo es quien completa el intercambio de token.

**Justificación Teórica 2 - Cumplimiento PSD2 Strong Customer Authentication (SCA):** El flujo Authorization Code permite redirección al IdP donde se implementan múltiples factores de autenticación (password + TOTP/biometría), cumpliendo requisitos de SCA que exigen 2 de 3 factores: conocimiento (password), posesión (móvil), inherencia

(huella/facial). Además, los tokens de acceso de corta duración (15 min) + refresh tokens rotativos minimizan riesgo de token theft.

**Alternativas Evaluadas:**

- **Implicit Flow:** Obsoleto, inseguro (tokens en URL)
- **Password Grant:** Expone credenciales al cliente, prohibido en banca
- **Client Credentials:** Solo para service-to-service, no para usuarios

---

**ADR-008: Patrón Saga para Transacciones Distribuidas**

**Decisión:** Implementar Saga Pattern (orquestación vía Kafka) para transferencias interbancarias

**Justificación Teórica 1 - Consistencia sin 2PC (Two-Phase Commit):** Las transferencias interbancarias involucran múltiples servicios: Account Service (debitar), Core Bancario (transferir), Notification Service (notificar). Usar 2PC distribuido sería inaceptable por bloqueo de recursos y latencia. Saga permite consistencia eventual mediante secuencia de transacciones locales, donde cada paso publica un evento que dispara el siguiente. Si un paso falla, se ejecutan compensating transactions (sagas reversas) para rollback.

**Justificación Teórica 2 - Tolerancia a Fallos y Resiliencia:** Al ser asíncrono y basado en eventos, si el Core Bancario está temporalmente indisponible, la saga se pausa y retry automáticamente (con backoff exponencial). Los estados intermedios se persisten en Kafka (duradero), permitiendo recovery sin pérdida de datos. Esto es crítico para transacciones financieras donde no se puede perder ni duplicar dinero.

**Alternativas Evaluadas:**

- **2PC/XA:** Bloqueo de recursos, inaceptable performance
- **Event Sourcing Completo:** Mayor complejidad, overkill

---

**ADR-009: CQRS (Command Query Responsibility Segregation)**

**Decisión:** Implementar CQRS para Movement Service (consultas de histórico)

**Justificación Teórica 1 - Optimización de Lecturas vs Escrituras:** El patrón CQRS separa modelos de escritura (Command) y lectura (Query). Para consultas de movimientos históricos, los usuarios realizan queries complejas (filtros por fecha, monto, categoría,

paginación). Usar el mismo modelo transaccional (PostgreSQL) para lecturas pesadas degradaría performance de escrituras críticas (transferencias). Con CQRS, las escrituras van a PostgreSQL (Command DB) y se proyectan a MongoDB (Query DB) optimizado para lecturas con índices específicos, mejorando performance de queries en 5-10x.

**Justificación Teórica 2 - Escalabilidad Independiente:** MongoDB puede tener múltiples read replicas escalables horizontalmente para servir consultas sin afectar la DB transaccional. Esto permite que miles de usuarios consulten movimientos simultáneamente mientras el sistema procesa transferencias críticas sin contención de locks o recursos.

**Alternativas Evaluadas:**

- **Sin CQRS:** Queries complejas en DB transaccional, degradación
- **Read Replicas SQL:** Menor flexibilidad que MongoDB para documentos

---

**ADR-010: GitOps con ArgoCD**

**Decisión:** Implementar GitOps para despliegue y gestión de infraestructura

**Justificación Teórica 1 - Infraestructura Declarativa y Versionada:** GitOps (acuñado por Weaveworks) usa Git como única fuente de verdad. Toda la infraestructura (K8s manifests, Helm charts, Terraform) está versionada en Git. ArgoCD monitorea el repo y sincroniza automáticamente el cluster con el estado deseado. Esto proporciona: audit trail completo (quién cambió qué y cuándo), rollback inmediato (git revert), y disaster recovery (recrear cluster desde Git), esencial para compliance SOX y auditorías.

**Justificación Teórica 2 - Automatización y Consistencia:** Elimina drift de configuración (diferencias entre ambientes) y reduce errores humanos en 80-90%. Los deployments son automáticos al hacer merge a main, con aprobaciones manuales para producción. Esto acelera time-to-market de semanas a horas mientras mejora calidad y estabilidad.

**Alternativas Evaluadas:**

- **CI/CD Tradicional (Jenkins):** Menor automatización, más manual
- **Helm Solo:** Sin reconciliación continua

---

## 6. Arquitectura de Seguridad y Onboarding

### 6.1 Zero Trust Security Model

**Implementación:**

| Capa | Control | Tecnología |
|---|---|---|
| **Perímetro** | DDoS Protection, WAF | AWS Shield Advanced, AWS WAF / Azure Front Door |
| **Red** | Network Policies, mTLS | Istio Service Mesh, Calico Network Policies |
| **Servicio** | Authentication, Authorization | OAuth2.0, JWT, OPA (Open Policy Agent) |
| **Datos** | Encryption at Rest & Transit | TLS 1.3, AES-256, Field-level encryption |
| **Identidad** | MFA, Biometrics | WebAuthn, TOTP, Facial Recognition |

### 6.2 Flujo de Onboarding con Reconocimiento Facial

**Herramientas Recomendadas:**

| Función | Herramienta | Justificación |
|---|---|---|
| **Facial Recognition** | **AWS Rekognition / Azure Face API** | **Precisión >99%, liveness detection integrado, compliance GDPR** |
| **Liveness Detection** | **SDK nativo (iProov, Onfido)** | **Previene spoofing con fotos/videos, requerimiento regulatorio** |
| **Document OCR** | **AWS Textract / Azure Form Recognizer** | **Extracción automática de DNI/pasaporte, validación de autenticidad** |
| **Biometric Auth** | **WebAuthn / FIDO2** | **Estándar W3C, sin contraseñas, phishing-resistant** |

### 6.3 Métodos de Autenticación Post-Onboarding

1. **Password + MFA (TOTP): Base, compatible con todos los dispositivos**
2. **Biometría Dispositivo: Huella dactilar (Touch ID/Fingerprint) o facial (Face ID/Face Unlock)**
3. **WebAuthn/FIDO2: Passwordless con llaves de seguridad (YubiKey) o biometría nativa**

**Recomendación: Implementar autenticación adaptativa basada en riesgo:**

- **Bajo riesgo (consulta de saldo): Biometría del dispositivo**
- **Medio riesgo (transferencia < $1,000): Password + MFA**
- **Alto riesgo (transferencia > $1,000, cambio de datos): Facial recognition + MFA**

---

## 7. Infraestructura Cloud, HA y DR

### 7.1 Arquitectura Kubernetes Multi-AZ

### 7.2 Alta Disponibilidad (HA)

| Componente | Estrategia HA | RTO | RPO |
|---|---|---|---|
| **Kubernetes** | Multi-AZ (3 AZs), Cluster Autoscaler | < 5 min | 0 |
| **PostgreSQL** | RDS Multi-AZ + Read Replicas | < 60s | < 5s |
| **MongoDB** | MongoDB Atlas Multi-Region | < 60s | < 5s |
| **Redis** | Redis Cluster (sharding + replicas) | < 30s | 0 |
| **Kafka** | MSK con 3 brokers (1 por AZ) | < 60s | 0 |
| **API Gateway** | Multi-AZ deployment | < 30s | 0 |

### 7.3 Disaster Recovery (DR)

**Estrategia:** Pilot Light + Warm Standby

| Elemento | Implementación |
|---|---|
| **Backup RDS** | Automated backups (retention 35 días), point-in-time recovery |
| **Backup MongoDB** | Continuous backups + snapshots diarios |
| **Backup Kafka** | MirrorMaker 2 a región secundaria |
| **Backup S3** | Cross-region replication a S3 Glacier |
| **Infraestructura** | Terraform state en S3 versionado + DynamoDB locking |
| **DR Region** | Región secundaria (ej. us-east-1 → us-west-2) |

| Elemento | Implementación |
|---|---|
| **RTO Target** | < 1 hora |
| **RPO Target** | < 15 minutos |

**Procedimiento de DR:**

1. Detectar desastre (health checks fallan)
2. Activar pipeline de DR (Terraform apply en región secundaria)
3. Restaurar bases de datos desde último snapshot
4. Actualizar DNS (Route53 failover)
5. Validar servicios
6. Comunicar a stakeholders

### 7.4 Auto-Healing

| Mecanismo | Implementación |
|---|---|
| **Pod Health** | Kubernetes liveness/readiness probes |
| **Node Health** | K8s Cluster Autoscaler + ASG health checks |
| **Service Health** | Istio circuit breakers, outlier detection |
| **Database** | RDS automatic failover |
| **Application** | Restart policies, pod disruption budgets |

---

## 8. Observabilidad y Monitoreo

### 8.1 Stack de Observabilidad

| Capa | Herramienta | Propósito |
|---|---|---|
| **Métricas** | Prometheus + VictoriaMetrics | Time-series metrics collection |
| **Visualización** | Grafana | Dashboards, alertas, SLO tracking |
| **Tracing** | Jaeger / AWS X-Ray | Distributed tracing |

| Capa | Herramienta | Propósito |
|---|---|---|
| **Logging** | Loki + Promtail / ELK Stack | Centralized logging |
| **APM** | Datadog / New Relic | Application Performance Monitoring |
| **Alerting** | PagerDuty / Opsgenie | Incident management |
| **SLO/SLI** | Sloth / Nobl9 | Service Level Objectives |

### 8.2 Golden Signals (SRE)

1. **Latency:** p50, p95, p99 por endpoint
    - Target: p95 < 500ms para transferencias
2. **Traffic:** Requests/second, bandwidth
    - Monitoreo por servicio y endpoint
3. **Errors:** Error rate, 5xx responses
    - Alerta si error rate > 0.1%
4. **Saturation:** CPU, memory, disk, network
    - Alerta si > 80% por 5 minutos

### 8.3 Alerting Strategy

| Nivel | Condición | Acción | Canal |
|---|---|---|---|
| **Critical** | Error rate > 1% o latencia p99 > 2s | Page on-call inmediatamente | PagerDuty (SMS/Call) |
| **Warning** | Error rate > 0.1% o CPU > 80% | Notificar en 15 min | Slack + Email |
| **Info** | Deployment completado, scaling event | Log para revisión | Slack |

---

## 9. Gestión de Costos

### 9.1 Estimación de Costos Mensuales (AWS)

| Servicio | Configuración | Costo Mensual |
|---|---|---|
| **EKS** | Control plane + 6 nodos (m5.xlarge) | $800 |
| **EC2 (Spot)** | 14 nodos spot (70% de cluster) | $1,200 |
| **RDS PostgreSQL** | db.r5.xlarge Multi-AZ + 2 read replicas | $1,500 |
| **MongoDB Atlas** | M30 cluster (3 nodes) | $800 |
| **ElastiCache Redis** | cache.r5.large cluster (6 nodes) | $600 |
| **MSK Kafka** | kafka.m5.large (3 brokers) | $900 |
| **API Gateway** | 10M requests/mes | $350 |
| **S3 + Glacier** | 10TB + archive | $400 |
| **CloudWatch + X-Ray** | Logs, métricas, tracing | $500 |
| **Datadog** | APM, logs, synthetics | $1,200 |
| **NAT Gateway** | 3 AZs | $150 |
| **Load Balancers** | 2 ALBs + 1 NLB | $100 |
| **Total** | | **$8,500/mes** |

### 9.2 Comparativa Fase 1 vs Fase 2

| Concepto | Fase 1 (PaaS) | Fase 2 (K8s) | Variación |
|---|---|---|---|
| **Total Mensual** | $5,100 | $8,500 | +67% |
| **Usuarios Soportados** | 100,000 | 1,000,000 | +1000% |
| **Transacciones/día** | 50,000 | 500,000 | +1000% |
| **Costo por Usuario** | $0.051 | $0.0085 | -83% |
| **Costo por Transacción** | $0.102 | $0.017 | -83% |

**Conclusión:** Aunque el costo absoluto aumenta 67%, el **costo unitario disminuye 83%** debido a economías de escala. Fase 2 es más eficiente para volúmenes altos.

### 9.3 Optimizaciones de Costo

1. **Spot Instances:** 70% de nodos worker con spot instances (ahorro 70% vs on-demand)
2. **Reserved Instances:** 1 año de commitment para RDS y nodos base (ahorro 30-40%)
3. **Auto-Scaling:** Scale to zero en ambientes de desarrollo/nocturno
4. **Storage Tiering:** S3 Intelligent-Tiering + Glacier para auditoría > 90 días
5. **Rightsizing:** Continuous monitoring con AWS Compute Optimizer
6. **Kafka Topic Retention:** Ajustar retención según necesidad (7-30 días)

---

## 10. Plan de Implementación

### 10.1 Roadmap de 6 Meses

| Mes | Fase | Actividades | Entregables | KPIs |
|---|---|---|---|---|
| **Mes 1** | **Foundation** | - Setup EKS cluster<br>- Configurar Istio<br>- CI/CD con ArgoCD<br>- Monitoring stack | - Cluster operativo<br>- Pipelines funcionando<br>- Dashboards básicos | - Cluster up<br>- GitOps activo |
| **Mes 2** | **Auth + Customer** | - Migrar Auth Service<br>- Migrar Customer Service<br>- OAuth2 + PKCE<br>- Database migration | - Auth en microservicio<br>- Customer service operativo | - 100% auth en K8s<br>- Latencia < 100ms |
| **Mes 3** | **Account + Transfer** | - Migrar Account Service<br>- Migrar Transfer Service<br>- Implementar Saga pattern<br>- Kafka integration | - 4 servicios migrados<br>- Transferencias funcionando | - Transacciones ACID<br>- Latencia < 500ms |

| Mes | Fase | Actividades | Entregables | KPIs |
|---|---|---|---|---|
| **Mes 4** | **Movement + Notification** | - Migrar Movement Service<br>- Migrar Notification Service<br>- CQRS implementation<br>- Multi-channel notifications | - 6 servicios operativos<br>- Notificaciones < 5s | - Queries < 200ms<br>- 2 canales activos |
| **Mes 5** | **Audit + Onboarding** | - Migrar Audit Service<br>- Migrar Onboarding Service<br>- Facial recognition integration<br>- WORM storage | - 8 servicios migrados<br>- Onboarding completo | - SLA 99.95%<br>- Facial >99% accuracy |
| **Mes 6** | **Optimization + DR** | - Performance tuning<br>- Chaos testing<br>- DR setup<br>- Security hardening<br>- Documentation | - Monolito retirado<br>- DR probado<br>- Docs completas | - SLA 99.95%<br>- RTO < 1h<br>- RPO < 15min |

**10.2 Métricas de Éxito**

| Categoría | Métrica | Target |
|---|---|---|
| **Performance** | Latencia p95 transferencias | < 500ms |
| **Availability** | SLA mensual | 99.95% |
| **Scalability** | Usuarios concurrentes | 10,000+ |
| **Deployment** | Frecuencia | 10+/día |
| **Lead Time** | Commit to production | < 2 días |
| **MTTR** | Mean time to recovery | < 30 min |
| **Error Rate** | 5xx responses | < 0.1% |

---

## 11. Conclusiones

### 11.1 Ventajas Clave de Fase 2

✅ **Escalabilidad Horizontal:** Soporta 10x más usuarios sin degradación de performance
✅ **Resiliencia Avanzada:** SLA 99.95% con auto-healing y DR probado
✅ **Velocidad de Desarrollo:** Equipos autónomos, deployments 100x más frecuentes
✅ **Observabilidad Completa:** Distributed tracing, métricas en tiempo real, alertas proactivas
✅ **Security by Design:** Zero trust, mTLS automático, policy as code, compliance PCI-DSS
✅ **Costo Eficiente:** 83% menor costo por transacción a gran escala
✅ **Future-Proof:** Arquitectura desacoplada, reusable, extensible

### 11.2 Compromisos y Riesgos

⚠️ **Complejidad Incrementada:** Requiere equipo con skills en Kubernetes, Istio, Kafka
⚠️ **Costo Inicial Mayor:** +67% vs Fase 1, pero ROI positivo a 12-18 meses
⚠️ **Curva de Aprendizaje:** 2-3 meses para estabilización del equipo
⚠️ **Operación Más Compleja:** Necesidad de SRE/DevOps dedicado

---

## 13. Enlace al Repositorio

El código fuente de la infraestructura (IaC) y este documento están disponibles en: 📎
**https://github.com/byros21/prueba-devsu**